Pandas for Python

1. The Basics

import pandas as pd

```
import pandas as pd
import numpy as np   # often used alongside pandas
```

We use 'pd' as a short alias so we don't have to type 'pandas' every time we call a function. It's a global convention.

Every tutorial, book, and Stack Overflow answer uses 'pd'. Using it makes your code instantly recognizable to any Python developer.

Never rename it to something else like 'import pandas as p'. You'll confuse teammates and none of the copy-pasted examples from the internet will work.

DataFrame — the core object

```
df = pd.DataFrame({
    "name":  ["Ana", "Bob", "Cara"],
    "age":   [25, 30, 22],
    "score": [88, 91, 76]
})
```

A DataFrame is a 2D table — like a spreadsheet or SQL table. It is the primary object in pandas. Almost everything you do in pandas starts with a DataFrame.

DataFrames let you store, filter, transform and analyze thousands of rows of mixed data (numbers, text, dates) all at once — far more powerful than plain Python lists or dicts.

If you only have one column of data, you might be working with a Series instead of a DataFrame. Use type(df) to check. Single brackets df['col'] return a Series; double brackets df[['col']] return a DataFrame.

Quick Overview Commands

```
df.head()        # first 5 rows
df.tail(3)       # last 3 rows
df.shape         # (rows, columns) as a tuple
df.info()        # column types + null counts
df.describe()    # statistics summary
df.columns       # list of column names
df.dtypes        # data type of each column
len(df)          # number of rows
```

Before doing any analysis, you need to understand what data you're working with — its size, types, and whether anything is missing.

✅ *df.info() is the single most useful first command. It tells you instantly which columns have missing data and whether types look correct (e.g. a 'date' column stored as text is a red flag).*

⚠️ *Run df.head() and df.info() immediately after loading any new dataset. Don't skip this step — most bugs come from wrong data types or unexpected NaN values you didn't know were there.*

---

2. Selecting Data

Select a Column

```
df["name"]            # returns a Series
df[["name", "age"]]   # returns a DataFrame
```

💡 *You rarely need all columns at once. Selecting only the columns you need makes code faster and easier to read.*

✅ *Selecting specific columns is essential for analysis — you pass columns to functions, filter on them, and build new ones from them.*

⚠️ *Single [ ] → Series. Double [[ ]] → DataFrame. Mixing them up is the #1 beginner mistake. If you pass a DataFrame to a function that expects a Series (or vice versa), you'll get confusing errors.*

iloc — Select by Position Number

```
df.iloc[0]        # first row
df.iloc[-1]       # last row
df.iloc[1:3]      # rows 1 and 2 (end EXCLUSIVE)
df.iloc[0, 1]     # row 0, column 1
```

💡 *iloc lets you select rows and columns by their integer position (0, 1, 2...) just like indexing a Python list.*

✅ *Useful when you don't know column names, when you want 'the first N rows', or when working with numeric indices programmatically.*

⚠️ *iloc slicing is EXCLUSIVE at the end — iloc[1:3] gives rows 1 and 2, not 3. This is standard Python slicing behaviour. Be careful when the index has been reset or sorted — position 0 might not be the 'first' record logically.*

loc — Select by Label

```
df.loc[0]                    # row with index label 0
df.loc[0, "age"]             # single cell
df.loc[0:2, "name":"score"]  # BOTH ends INCLUSIVE
```

💡 *loc lets you select by the actual label of rows and columns — useful when your index has meaningful labels like dates, IDs, or names.*

✅ *loc is safer for writing to the DataFrame (df.loc[condition, 'col'] = value) and is the correct way to avoid the SettingWithCopyWarning.*

⚠️ *loc slicing is INCLUSIVE on both ends — loc[0:2] gives rows 0, 1, AND 2. This is different from iloc and normal Python. Always use .loc for setting values, never df[condition]['col'] = value.*

Filter with Conditions

```
df[df["age"] > 24]
df[(df["age"] > 24) & (df["score"] > 85)]
df[(df["age"] < 23) | (df["score"] > 90)]
df[df["name"].isin(["Ana", "Bob"])]
df.query("age > 24 and score > 85")
```

💡 *Filtering is how you answer questions like 'show me all customers over 30' or 'which products scored above 90%'. It's the foundation of data analysis.*

✅ *Boolean filtering creates a mask of True/False values and applies it to the DataFrame instantly — no loops, no list comprehensions, just one clean line.*

⚠️ *Always wrap each condition in its own parentheses when combining with & or |. Never use Python's 'and'/'or' keywords on Series — they raise a ValueError. For readable filters, prefer query().*

---

3. Cleaning Data

Missing Values (NaN)

```
df.isnull()                          # True where NaN
df.isnull().sum()                    # count per column
df.dropna()                          # remove rows with NaN
df.dropna(subset=["age"])            # only if age is NaN
df.fillna(0)                         # replace NaN with 0
df["age"].fillna(df["age"].mean())   # fill with average
```

💡 *Real-world data almost always has gaps — people skip survey questions, sensors malfunction, imports partially fail. NaN marks those gaps.*

✅ *Knowing where your missing data is prevents silent bugs. A mean() on a column with NaN still works (pandas skips NaN) but a count or a merge can behave unexpectedly if you don't clean first.*

⚠️ *Never check NaN with == : df['col'] == None or df['col'] == np.nan both return False because NaN != NaN by definition. Always use .isnull() or .notna(). Choose dropna() when you can afford to lose rows; fillna() when you can't.*

Rename, Drop, Change Types

```
df.rename(columns={"name": "full_name"})
df.drop(columns=["score"])
df.drop(index=[0, 2])
df["age"] = df["age"].astype(float)
df["date"] = pd.to_datetime(df["date"])
```

💡 *Column names from raw data are often messy (spaces, uppercase, abbreviations). Types are often wrong — dates stored as strings, numbers stored as objects.*

✅ *Clean names and correct types make every subsequent operation work correctly. datetime columns unlock .dt.year, .dt.month etc. Numeric columns unlock math operations.*

⚠ *Most pandas methods return a new DataFrame — they do NOT modify the original. Always reassign: df = df.drop(...) or use inplace=True. Forgetting this is a very common beginner mistake.*

Remove Duplicates

```
df.duplicated()                      # find them
df.drop_duplicates()                 # remove them
df.drop_duplicates(subset=["name"])  # by specific column
```

💡 *Duplicate rows can skew every calculation — averages, counts, totals all become wrong if the same record appears twice.*

✅ *drop_duplicates() is a one-liner fix. The subset parameter lets you define what 'duplicate' means — e.g. same name even if other columns differ.*

⚠ *Always check df.duplicated().sum() early. A dataset with 1000 rows might actually have 200 duplicates. Decide which copy to keep using the keep parameter: keep='first', keep='last', or keep=False to drop all.*

---

4. Analyzing Data

Basic Statistics

```
df["score"].mean()
df["score"].median()
df["score"].max()  /  df["score"].min()
df["score"].std()
df["score"].sum()
df["score"].count()   # non-NaN count
```

💡 *Statistics summarize large datasets into single numbers you can understand and compare — essential for any data analysis or reporting.*

✅ *These functions automatically skip NaN values, so you get meaningful results even on messy data. They also work on an entire DataFrame at once: df.mean() gives mean of all numeric columns.*

⚠ *Use .count() (non-NaN rows) rather than len(df) when your column has missing values — otherwise you overcount. Use .median() instead of .mean() when your data has outliers that would skew the average.*

value_counts() and unique()

```
df["name"].value_counts()   # frequency of each value
df["name"].nunique()        # count of unique values
df["name"].unique()         # array of unique values
```

💡 *Before analyzing a categorical column you need to know what values exist in it, how many there are, and how often each appears.*

✅ *value_counts() is perfect for spotting typos and inconsistencies — e.g. 'male', 'Male', 'MALE' all showing up as separate categories.*

⚠ *value_counts() returns a Series sorted by frequency (highest first). Pass normalize=True to get percentages instead of counts. Use nunique() to quickly check if an 'ID' column really has all unique values.*

groupby() — The Most Powerful One

```
df.groupby("team")["score"].mean()
df.groupby("team").agg({
    "score": ["mean", "max"],
    "age":   "median"
})
df.groupby("team").size()
```

💡 *groupby() answers questions like 'what is the average score per team?' or 'how many orders per customer?'. It's the pandas equivalent of SQL's GROUP BY.*

✅ *Instead of writing loops to calculate stats per group, groupby() does it in one line. It's one of the most-used functions in real data work — dashboards, reports, and ML feature engineering all use it constantly.*

⚠️ *groupby() returns a GroupBy object, not a DataFrame. After .agg() or .mean() etc., call .reset_index() to turn it back into a normal DataFrame with clean column names you can filter and merge as usual.*

---

5. Transforming Data

Add / Modify Columns

```
df["country"] = "India"
df["score_pct"] = df["score"] / 100
df["grade"] = df["score"].apply(lambda x: "A" if x >= 90 else "B")
df["pass"] = np.where(df["score"] >= 50, "Pass", "Fail")
```

💡 *Raw data rarely has all the columns you need. You almost always need to engineer new features — ratios, labels, flags, bucketed values — before analysis or ML.*

✅ *Creating derived columns is vectorized (instant on millions of rows). np.where() is the fastest way to create conditional columns. apply() is flexible for complex logic.*

⚠️ *Prefer vectorized operations (df['a'] + df['b']) over apply(lambda) whenever possible — they're 10-100x faster. Use np.where() for simple if-else conditions instead of apply.*

apply() — Custom Function

```
df["name"].apply(str.upper)
df["score"].apply(lambda x: x * 2)
def classify(x):
    if x >= 90: return 'A'
    elif x >= 75: return 'B'
    else: return 'C'
df["grade"] = df["score"].apply(classify)
```

💡 *apply() lets you run any custom Python function on each element of a column. It's the escape hatch when no built-in vectorized function exists for your logic.*

✅ *Keeps your code clean by separating logic into a named function. Great for complex multi-condition transformations that would be unreadable as a lambda.*

⚠️ *apply() is slow because it loops row by row in Python. Always ask first: can I do this with arithmetic, .str methods, .dt methods, or np.where? Only fall back to apply() when those don't cover it.*

merge() — Join Two DataFrames

```
pd.merge(df1, df2, on="id")              # inner join
pd.merge(df1, df2, on="id", how="left")  # left join
pd.merge(df1, df2, on="id", how="outer") # keep all rows
```

Real data lives in multiple tables (customers, orders, products). merge() combines them — just like SQL JOIN. You almost always need to merge before doing any meaningful analysis.

merge() automatically matches rows by the key column and handles all the joining logic. The how parameter gives you full control over which rows to keep from each side.

When both DataFrames share column names other than the key, pandas adds '_x' and '_y' suffixes. Use the suffixes parameter to give them meaningful names. Always check the shape before and after merging — unexpected row count means duplicate keys.

concat() — Stack DataFrames

```
pd.concat([df1, df2], ignore_index=True)  # stack rows
pd.concat([df1, df2], axis=1)             # add columns side by side
```

concat() is used when you have data split across multiple files (e.g. one CSV per month) and need to combine them into one table.

ignore_index=True gives you a clean 0,1,2... index instead of repeating 0,1,2,0,1,2. axis=1 is useful for combining computed columns with the original DataFrame.

If the column names don't match between the two DataFrames, concat creates NaN-filled columns for the missing side. Always verify df.columns for both DataFrames before concatenating.

---

6. Reading & Saving Files

CSV — Most Common Format

```
df = pd.read_csv("data.csv")
df = pd.read_csv("data.csv", sep=";", usecols=["a","b"], nrows=100)
df.to_csv("output.csv", index=False)
```

CSV (Comma Separated Values) is the universal format for tabular data. Every database, Excel, and data tool can export and import it.

read_csv() is incredibly flexible — handles different separators, encodings, date parsing, and type inference automatically. usecols and nrows let you load only what you need, saving memory on large files.

Always use index=False when saving. Without it, pandas adds an extra column '0,1,2...' that you almost never want. If you get encoding errors, try pd.read_csv('file.csv', encoding='latin-1').

Excel

```
df = pd.read_excel("data.xlsx", sheet_name="Sheet1")
df.to_excel("output.xlsx", index=False)
```

Many businesses still distribute data in Excel format. read_excel() lets you work with it directly in pandas without any manual conversion.

✅ *sheet_name can be a name (string), a number (0 for first sheet), or None to get all sheets as a dict. You can also write multiple sheets using ExcelWriter.*

⚠️ *Excel support requires the openpyxl library. Install it with: pip install openpyxl. For .xls (old format) use xlrd instead. Excel files are slower to read than CSV — convert to CSV first for large files.*

JSON

```
df = pd.read_json("data.json")
df.to_json("output.json", orient="records")
```

💡 *JSON is the standard format for web APIs. When you pull data from a REST API, it's almost always JSON.*

✅ *orient='records' produces a clean list of objects format [{col: val}, ...] which is what most web applications expect.*

⚠️ *Nested JSON (objects inside objects) won't load cleanly into a flat DataFrame. Use pd.json_normalize() to flatten nested structures first.*

---

7. Special Cases & Edge Situations

Chained Assignment Warning

Modifying a slice of a DataFrame with chained indexing — the most confusing beginner error.

```
# WRONG — may silently NOT work:
df[df["age"] > 25]["score"] = 100

# CORRECT — always use .loc:
df.loc[df["age"] > 25, "score"] = 100
```

💡 *When you chain two [ ][ ] operations, pandas can't tell if you're working on the original or a temporary copy — so the assignment may be silently ignored.*

✅ *Using .loc[] directly guarantees you're modifying the original DataFrame. It also makes your intent crystal clear to other developers reading your code.*

⚠️ *Rule: whenever you SET values based on a filter, always use df.loc[condition, 'column'] = value. Never chain two bracket operations for assignment. Enable warnings with pd.options.mode.chained_assignment = 'raise' during development.*

Integer Columns Become Float After fillna()

```
df['age'] = [25, None, 30]
# df['age'].dtype is float64 because of the None!
df['age'].fillna(0, inplace=True)
df['age'] = df['age'].astype(int)  # convert back
```

💡 *Pandas can't store NaN in an integer column (NaN is a float concept in NumPy). So any integer column with even one missing value becomes float64.*

✅ *Understanding this prevents surprises like IDs turning into '1001.0' instead of '1001'.*

⚠ *After fillna(), always call .astype(int) to restore integer type. Alternatively, use pandas nullable integer type: df['age'] = df['age'].astype('Int64') — capital I — which supports NaN natively.*

reset_index() After Filtering

```
filtered = df[df["age"] > 25]
# Index: 1, 3, 4  (gaps from original positions)

filtered = filtered.reset_index(drop=True)
# Index: 0, 1, 2  (clean again)
```

💡 *After filtering, the original row labels (index) are preserved. Rows that matched keep their old numbers, leaving gaps in the index sequence.*

✅ *A clean 0,1,2... index prevents confusing bugs in loops, iloc access, and when concatenating DataFrames together.*

⚠ *Always use drop=True — otherwise the old index becomes an extra column. Make reset_index(drop=True) a habit whenever you filter or sort a DataFrame that you'll use further.*

Boolean Operators: & and | not 'and'/'or'

```
# WRONG:
df[df["age"] > 25 and df["score"] > 80]  # ValueError!

# CORRECT:
df[(df["age"] > 25) & (df["score"] > 80)]
df[(df["age"] < 22) | (df["score"] > 90)]
```

💡 *Python's 'and'/'or' keywords work on single True/False values, but a pandas condition produces a whole Series of True/False — which 'and' can't handle.*

✅ *& and | apply the logic element-by-element across the entire Series, which is exactly what you want. The parentheses force correct operator precedence.*

⚠ *Parentheses are REQUIRED around each condition — not optional. Without them, Python's operator precedence applies > before &, giving wrong results silently. When conditions get complex, switch to df.query() for readability.*

Views vs Copies — Use .copy()

```
df2 = df           # view — modifying df2 changes df!
df2 = df.copy()   # independent copy — safe to modify
```

💡 *Pandas sometimes returns a 'view' into the original data rather than a new object. Modifying the view unexpectedly changes the original.*

✅ *df.copy() creates a fully independent DataFrame. This is the safe default whenever you want to work on a subset or modified version without touching the source.*

⚠ *When in doubt, use .copy(). The tiny memory cost is worth avoiding mysterious bugs where data changes 'by itself'. This is especially important inside functions that receive a DataFrame as a parameter.*

apply() is Slow — Use Vectorization

```
# Slow (loops in Python):
```

```
df['score'].apply(lambda x: x * 2)

# Fast (vectorized — C speed):
df['score'] * 2
df['name'].str.upper()     # string vectorized
df['date'].dt.year         # datetime vectorized
```

💡 *apply() iterates row-by-row in Python, which is slow. Vectorized operations run in compiled C code under the hood and process all rows at once.*

✅ *Vectorized operations can be 10-100x faster on large DataFrames. They also result in much shorter, more readable code.*

⚠️ *Before writing apply(), ask: is there a built-in pandas method? Use .str. for strings, .dt. for datetimes, arithmetic operators for math. Only use apply() for genuinely complex logic with no vectorized equivalent.*

squeeze() — Collapse Single-Column DataFrame to Series

```
result = df[["score"]]   # DataFrame (double brackets)
result.squeeze()         # → Series

# groupby returns DataFrame, squeeze to Series:
totals = df.groupby("team")[["score"]].sum()
totals.squeeze()

s = pd.Series([42])
s.squeeze()              # → scalar: 42
```

💡 *Some functions always return a DataFrame even when there's only one column. squeeze() reduces it to a simpler Series or scalar, making it easier to work with downstream.*

✅ *Saves you from having to write ['col'] everywhere after a groupby or read_csv. Cleaner code when you know you only have one column.*

⚠️ *squeeze() only works if there is exactly ONE column (or row). On a multi-column DataFrame it silently does nothing — no error. Always verify with df.shape first. For column axis only: df.squeeze(axis='columns').*

Duplicate Column Names After merge()

```
pd.merge(df1, df2, on="id")
# If both have 'score': score_x (df1) and score_y (df2)

pd.merge(df1, df2, on='id', suffixes=('_before', '_after'))
```

💡 *When two DataFrames being merged share column names (other than the join key), pandas must distinguish them — so it adds suffixes automatically.*

✅ *The suffixes parameter lets you give them meaningful names immediately rather than having to rename after the merge.*

⚠️ *Always check df.columns after a merge. Unexpected _x / _y suffixes mean you had overlapping names. Use suffixes= to name them at merge time, or rename columns in one DataFrame before merging.*

---

8. Tricky Questions & Gotchas

These are the questions that trip up even experienced users.

Q: What is the difference between .loc and .iloc?

```
.loc["a"]     # uses INDEX LABEL — whatever the index contains
.iloc[0]      # uses POSITION — always 0, 1, 2...
```

💡 *WHY: After filtering, the index labels stay the same but positions shift. If you filtered rows 0, 2, 4 — iloc[1] is now the row that was originally row 2.*

✅ *WHY HELPFUL: loc is safer when your index has meaningful labels (dates, IDs). iloc is safer when you always want 'the Nth row' regardless of labels.*

⚠️ *HOW TO HANDLE: After filtering, call reset_index(drop=True) to sync labels and positions. Use loc for setting values, iloc for positional slicing.*

Q: Why does df.drop() not change my DataFrame?

```
df.drop(columns=['age'])            # returns NEW df — original unchanged!
df = df.drop(columns=['age'])       # correct: reassign
df.drop(columns=['age'], inplace=True)  # or inplace=True
```

💡 *WHY: Pandas follows immutability by default — most methods return a new object to prevent accidental data loss.*

✅ *WHY HELPFUL: This means your original data is always safe. You can try operations without fear of breaking your source DataFrame.*

⚠️ *HOW TO HANDLE: Either reassign (df = df.method()) or use inplace=True. Reassigning is generally preferred as it's clearer and works in all contexts.*

Q: Why does NaN == NaN return False?

```
np.nan == np.nan    # False — NaN is never equal to itself
df['age'] == None   # also wrong

# CORRECT:
df['age'].isnull()  # always use this
```

💡 *WHY: NaN (Not a Number) is defined by the IEEE floating-point standard to be unordered — it cannot be compared to anything, including itself.*

✅ *WHY HELPFUL: This strict rule prevents silent bugs where two 'unknown' values get incorrectly treated as equal.*

⚠️ *HOW TO HANDLE: Always use .isnull() or .notna() to check for missing values. Never use == None or == np.nan.*

Q: Why does pd.concat produce columns full of NaN?

```
# DataFrames with different column names:
df1 = pd.DataFrame({'A': [1,2]})
df2 = pd.DataFrame({'B': [3,4]})
pd.concat([df1, df2])   # A and B both exist, filled with NaN

# Fix: make column names match first, or use axis=1
```

💡 *WHY: When concatenating rows, pandas creates all unique columns from both DataFrames. Where a DataFrame doesn't have that column, NaN fills the gap.*

✅ *WHY HELPFUL: This behaviour is actually useful — it tells you exactly where the data mismatch is, rather than silently dropping columns.*

⚠️ *HOW TO HANDLE: Rename columns before concatenating so they match. Check result shape and use df.isnull().sum() after concat to catch gaps.*

Q: Why is iterrows() bad — when should I never use it?

```
# Slow — loops in Python:
for i, row in df.iterrows():
    df.loc[i, 'x'] = row['a'] + row['b']   # very slow

# Fast — vectorized:
df['x'] = df['a'] + df['b']
```

💡 *WHY: iterrows() processes one row at a time in Python (slow). Vectorized operations run in compiled C code on all rows simultaneously (fast).*

✅ *WHY HELPFUL: On a 100,000-row DataFrame, vectorized is typically 50-200x faster. It also produces cleaner, shorter code.*

⚠️ *HOW TO HANDLE: Replace loops with arithmetic, .str, .dt, .apply(), or np.where(). Only use iterrows() as a last resort for logic that cannot be vectorized.*

Q: What happens if I sort a DataFrame — does it affect iloc?

```
df = df.sort_values('score')
df.iloc[0]   # now the row with LOWEST score, not original row 0

# Fix: reset index after sorting
df = df.sort_values('score').reset_index(drop=True)
```

💡 *WHY: sort_values() reorders rows but keeps their original index labels. iloc[0] is always the first physical row, which changes after sorting.*

✅ *WHY HELPFUL: Understanding this prevents bugs where you think you're accessing 'the top scorer' but you're actually accessing the original first row.*

⚠️ *HOW TO HANDLE: Always reset_index(drop=True) after sorting if you plan to use iloc. Use df.nlargest(n, 'col') or df.nsmallest(n, 'col') to get top/bottom rows cleanly.*

Q: Why do groupby results have a weird multi-level index?

```
result = df.groupby('team').agg({'score': ['mean', 'max']})
# Columns are now: ('score', 'mean') and ('score', 'max')

# Flatten multi-level columns:
result.columns = ['score_mean', 'score_max']
result = result.reset_index()
```

💡 *WHY: When you agg with multiple functions, pandas creates a MultiIndex on columns to represent both the original column name and the function applied.*

✅ *WHY HELPFUL: The structure is actually informative — you can see exactly which aggregation produced each column.*

⚠️ *HOW TO HANDLE: Flatten column names with result.columns = ['_'.join(c) for c in result.columns] then reset_index() to get a clean DataFrame.*

Happy coding! 🐼 — Full docs at pandas.pydata.org